

Что входит в JUnit 5

JUnit Platform	Находит движки и связывает запуск с IDE, Maven и Gradle.
Jupiter API	@Test, lifecycle, assertions, parameterized tests, extensions.
Jupiter Engine	Находит и выполняет тесты, написанные на Jupiter API.

Maven Surefire обычно ищет классы *Test, *Tests, Test* в src/test/java.

Структура Arrange → Act → Assert

```
@Test
void discountIsApplied() {
    // Arrange: данные и зависимости
    PriceCalculator calculator = new PriceCalculator();

    // Act: одно проверяемое действие
    int actual = calculator.applyDiscount(100, 20);

    // Assert: наблюдаемый результат
    assertEquals(80, actual);
}
```

Название теста описывает поведение. Один тест — один сценарий, но связанных assertions может быть несколько.

Основные assertions

```
assertEquals(expected, actual);
assertNotEquals(unexpected, actual);
assertTrue(condition);    assertFalse(condition);
assertNull(value);        assertNotNull(value);
assertSame(expectedRef, actualRef);
assertArrayEquals(expectedArray, actualArray);
```

Порядок: у JUnit сначала expected, затем actual. Сообщение передаётся последним аргументом.

assertAll: связанные поля

```
assertAll("Поля пользователя",
    () -> assertEquals("Ivan", user.name()),
    () -> assertEquals(28, user.age()),
    () -> assertTrue(user.active())
);
```

Обычный assertion останавливает тест на первой ошибке. assertAll выполняет все переданные лямбды и собирает связанные расхождения в один отчёт.

Сообщения об ошибках

```
assertEquals(expected, actual,
    () -> "Цена заказа " + orderId);
```

Supplier<String> строит сообщение только при падении. Добавляйте бизнес-контекст, а не повтор expected/actual.

Частые ошибки

- Перепутаны expected и actual.
- assertTrue(expected.equals(actual)) вместо типового assertEquals.
- Проверяется приватная реализация, а не результат для пользователя.
- Один тест проверяет несколько несвязанных сценариев.
- Есть действие, но нет meaningful assertion.
- Тест зависит от порядка выполнения соседних тестов.

Минимальный запуск

```
# все тесты
./mvnw test

# один класс / один метод
./mvnw -Dtest=JUnitTasksTest test
./mvnw -
Dtest=JUnitTasksTest#task01_basicAssertionsAndAssertAll
test
```

Жизненный цикл теста

@BeforeAll	Один раз до всех тестов класса.
@BeforeEach	Перед каждым тестом: новая fixture.
@AfterEach	После каждого теста, даже упавшего.
@AfterAll	Один раз после класса.

```
ShoppingCart cart;

@BeforeEach
void setUp() {
    cart = new ShoppingCart();
}
```

По умолчанию JUnit создаёт новый экземпляр тестового класса на каждый тест. Всё равно явно обновляйте изменяемую fixture в @BeforeEach.

@CsvSource: таблица простых значений

```
@ParameterizedTest(name = "{0} - {1}% = {2}")
@CsvSource({
    "100, 10, 90",
    "250, 20, 200",
    "80, 100, 0"
})
void discountCases(int price, int discount,
    int expected) {
    assertEquals(expected,
        calculator.priceAfterDiscount(price, discount));
}
```

Хорош для небольшой читаемой таблицы. Следите за преобразованием типов, кавычками, null и разделителями.

@MethodSource: объекты и сложные случаи

```
static Stream<Arguments> orderCases() {
    return Stream.of(
        Arguments.of(new Order(List.of()), 0),
        Arguments.of(new Order(List.of(10, 20)), 30)
    );
}

@ParameterizedTest(name = "order #{index} → {1}")
@MethodSource("orderCases")
void total(Order order, int expected) {
    assertEquals(expected, order.total());
}
```

Provider обычно static и возвращает Stream<Arguments>. При PER_CLASS может быть нестатическим.

Изоляция важнее удобства

Плохо

Тест В рассчитывает, что тест А уже создал данные.

@TestMethodOrder не лечит зависимость от состояния — он только маскирует её.

Хорошо

Каждый тест сам создаёт данные и очищает свои ресурсы.

@ValueSource: один простой аргумент

```
@ParameterizedTest(name = "id={0}")
@ValueSource(ints = {1, 2, 10, 999})
void positiveIdIsValid(int id) {
    assertTrue(id > 0);
}
```

Поддерживает строки, числа, boolean, char и Class. Для null/пустых значений есть @NullSource, @EmptySource, @NullAndEmptySource.

Как выбрать source

Данные	Источник
Один int/String	@ValueSource
Несколько простых колонок	@CsvSource
Enum	@EnumSource
DTO, коллекции, сложная подготовка	@MethodSource
Переиспользуемый provider	@ArgumentsSource

Параметризация не заменяет сценарии

Объединяйте случаи с одинаковой логикой. Если подготовка, действие или ожидаемое поведение различаются — отдельные тесты читаются лучше универсального метода с ветвлениями.

Проверка исключения

```
IllegalArgumentException error = assertThrows(
    IllegalArgumentException.class,
    () -> calculator.priceAfterDiscount(100, 101)
);

assertEquals("discount must be between 0 and 100",
    error.getMessage());
```

`assertThrows` возвращает пойманное исключение, поэтому можно проверить `message`, `cause` и дополнительные поля. Ручной `try/catch` с `boolean`-флагом не нужен.

Тип исключения

<code>assertThrows(Base.class)</code>	Принимает <code>Base</code> и любой подкласс.
<code>assertThrowsExactly(Base.class)</code>	Требует строго <code>Base</code> .
<code>assertDoesNotThrow(...)</code>	Контракт — действие не падает; может вернуть результат.

Выбирайте точность по контракту. Слишком общий `Exception.class` способен принять неправильный дефект.

Timeout

```
assertTimeout(Duration.ofMillis(300),
    () -> calculator.slowOperation());

String result = assertTimeout(Duration.ofMillis(300),
    () -> calculator.slowOperation());
```

`assertTimeout` ждёт завершения и затем сравнивает время. Код выполняется в текущем потоке.

```
assertTimeoutPreemptively(Duration.ofMillis(300), action);
```

Preemptive-вариант запускает действие отдельно и пытается прервать его. Это опасно для `ThreadLocal`, транзакций и `security context`.

@Timeout и assertTimeout

<code>@Timeout(1)</code>	Защитный предел метода/класса. Удобен от зависаний.
<code>assertTimeout(...)</code>	Явная часть контракта конкретной операции.

Можно использовать оба: `assertion` проверяет требование 300 мс, аннотация в 1 секунду страхует весь тест.

Assumptions: тест неприменим

```
String env = System.getProperty("test.env", "local");

assumeTrue("local".equals(env));

assertEquals("local-result", service.call());
```

Если `assumption` ложна, тест помечается **skipped**, а не **passed**. Это подходит для ОС, окружения или доступности внешней системы — не для бизнес-правил.

Не пытайте assumption и assertion

- **Assumption:** «этот тест можно выполнять здесь?»
- **Assertion:** «система повела себя правильно?»

Если неверная бизнес-цена приводит к **skipped**, дефект исчезает из отчёта. Это должна быть ошибка `assertion`.

Cleanup должен сработать при падении

```
@AfterEach
void cleanUp() {
    repository.delete(createdId);
}
```

Для локального ресурса также подходят `try/finally` и `try-with-resources`. Cleanup не должен скрывать исходную ошибку теста.

Диагностика зависшего теста

1. Определите, где зависание: `setup`, действие, `assertion` или `cleanup`.
2. Сохраните `method`, `URL`, `correlation id` и безопасные параметры.
3. Различите медленный продукт и ошибочный лимит теста.
4. Не лечите нестабильность бесконечным увеличением `timeout`.

@Nested: контекст сценария

```
@Nested
@DisplayName("Пустая корзина")
class EmptyCart {
    @Test
    void totalIsZero() {
        assertEquals(0, cart.total());
    }
}

@Nested
class FilledCart {
    @BeforeEach
    void addItem() { cart.add(10); cart.add(20); }

    @Test
    void sumsItems() { assertEquals(30, cart.total()); }
}
```

Nested-класс группирует тесты общей бизнес-предпосылкой и может иметь собственный @BeforeEach.

Понятные имена

```
@DisplayName("Скидка не может превышать 100%")
@Test
void excessiveDiscountIsRejected() { ... }

@ParameterizedTest(name = "[{index}] {0} → {1}")
```

Имя в отчёте должно позволять понять сценарий без чтения тела и быстро найти проблемный набор данных.

Dynamic tests

```
@TestFactory
Stream<DynamicTest> absoluteValues() {
    return Stream.of(-2, -1, 0, 1, 2)
        .map(value -> dynamicTest(
            "abs(" + value + ") >= 0",
            () -> assertTrue(Math.abs(value) >= 0)));
}
```

Factory возвращает тесты, а не выполняет assertions сама. Для каждого generated test не вызывается отдельный @BeforeEach.

Dynamic или parameterized?

Parameterized	Одна тестовая логика и таблица входов; обычный lifecycle на invocation.
Dynamic	Количество/структура тестов формируются во время выполнения.

Для обычной таблицы примеров выбирайте @ParameterizedTest. Dynamic нужен, когда тесты действительно генерируются.

Tags и выбор набора

```
@Tag("smoke")
@Test
void serviceIsAvailable() { ... }
```

smoke regression api ui slow

Теги описывают назначение набора, а не случайные детали. Фильтрацию настраивают Surefire/Gradle/IDE.

@Disabled

Отключённый тест не проверяет продукт. В рабочем проекте указывайте причину и задачу на восстановление:

```
@Disabled("BUG-142: сервис возвращает 500")
```

В учебных заготовках пустой @Disabled служит переключателем текущего задания.

Extensions

```
@ExtendWith(TimingExtension.class)
class ServiceTest { ... }

class TimingExtension
    implements BeforeEachCallback,
               AfterEachCallback { ... }
```

Возможности: lifecycle callbacks, ParameterResolver, TestWatcher, execution conditions, обработка ошибок.

Что прятать в extension

- Логирование и измерение времени.
- Выдачу инфраструктурных объектов.
- Скриншот/логи после падения.
- Технический cleanup.

Не прячьте бизнес-действие и ключевые assertions: тест должен оставаться читаемым.

Пирамида ответственности теста

Fixture	Минимальные данные и зависимости, нужные сценарию.
Action	Одно наблюдаемое бизнес-действие.
Assertions	Строгий контракт результата и значимых эффектов.
Cleanup	Удаление созданного независимо от исхода assertion.
Diagnostics	Данные, достаточные для воспроизведения падения.

Наблюдаемое поведение

Хрупко

Проверять private-метод, количество внутренних шагов и случайный порядок.

Устойчиво

Проверять результат, публичный контракт и значимый внешний эффект.

Interaction verification через Mockito оправдана, только если сам вызов зависимости входит в контракт.

Повторяемые данные

- Создавайте уникальные данные, если тесты идут параллельно.
- Сохраняйте seed/значение случайных данных в отчёте.
- Не используйте текущее время без Clock или допустимого диапазона.
- Не полагайтесь на заранее существующую запись.
- Каждый тест очищает только созданные им ресурсы.

Flaky-тест — это дефект

Retry допустим как временная диагностическая мера, но не как постоянная маскировка гонки, общего состояния, неправильного ожидания или нестабильного окружения.

Сначала классифицируйте: product defect, test defect, data defect или environment defect.

Checklist перед review

1. Имя описывает условие и ожидаемое поведение.
2. Тест выполняет один самостоятельный сценарий.
3. Fixture минимальна и создаётся независимо.
4. Нет зависимости от порядка запуска.
5. Выбран подходящий assertion и не перепутаны expected/actual.
6. Parameterized source соответствует типу данных.
7. Исключение проверяется через assertThrows.
8. Timeout обоснован и не маскирует проблему.
9. Cleanup выполняется после падения.
10. Отчёт содержит данные для воспроизведения.

Алгоритм разбора падения

1. Прочитайте имя сценария, expected, actual и stack trace.
2. Проверьте входные данные и окружение запуска.
3. Запустите один метод независимо от остальных.
4. Определите, упали setup, action, assertion или cleanup.
5. Сравните контракт с фактическим поведением продукта.
6. Зафиксируйте воспроизводимый минимальный пример.

```
./mvnw -Dtest=ClassName#methodName test
```

Когда какой тест

Один пример	@Test
Одна логика, много данных	@ParameterizedTest
Генерация структуры в runtime	@TestFactory
Общий бизнес-контекст	@Nested
Общая техническая механика	Extension

Достаточный уровень middle+

Вы можете объяснить lifecycle, выбрать источник параметров, проверить исключение и время, организовать независимые сценарии, обеспечить cleanup и локализовать причину падения без привязки теста к деталям реализации.